

next_permutation 정리

현재 순열을 사전순 다음 순열로 바꾸는 알고리즘입니다.

사용처 (Where)

- 모든 순서를 시도해야 하는 문제
- 순열 완전탐색
- 배치 경우의 수 확인

방법 (How to Use)

- 처음에 정렬된 상태에서 시작하면 모든 순열 순회 가능
- 다음 순열이 있으면 true
- 마지막 순열이면 false
- 원본 순서를 직접 바꿈

기본 정보

- 필요 헤더: `#include <algorithm>`
- 기본 선언: `next_permutation(v.begin(), v.end());`

왜 써야 할까? (Why)

- 순열 생성을 직접 구현하지 않아도 됨
- 완전탐색 문제에서 빠르게 시도 가능
- 사전순 순열 흐름을 쉽게 다룸

주요 함수

함수/문법	의미
<code>next_permutation</code>	다음 순열
<code>prev_permutation</code>	이전 순열
<code>sort</code>	전체 순열 시작 전 정렬
<code>do-while</code>	첫 순열 포함 순회

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3};

    do {
        for (int x : v) cout << x;
        cout << '\n';
    } while (next_permutation(v.begin(), v.end()));
}
```

next_permutation 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
한 번 호출	$O(n)$
전체 순열	$O(n! * n)$
메모리	거의 없음

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	next_permutation 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 세 숫자로 만들 수 있는 순서 출력

정렬 후 do-while 구조를 사용하면 첫 순열부터 마지막 순열까지 모두 처리합니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {3, 1, 2};
    sort(v.begin(), v.end());

    do {
        for (int x : v) cout << x << ' ';
        cout << '\n';
    } while (next_permutation(v.begin(), v.end()));
}
```

처음 배울 때 자주 하는 실수

- 처음 정렬하지 않으면 일부 순열만 볼 수 있음
- n 이 조금만 커져도 $n!$ 이라 매우 커짐
- 중복 원소가 있으면 중복을 고려한 순열 흐름이 됨

비슷한 항목과 차이

- prev_permutation은 이전 순열
- DFS 백트래킹은 조건 가지치기에 좋음
- combination은 조합이라 순서 개념이 다름

한 줄 정리: next_permutation은(는) '모든 순서를 시도해야 하는 문제' 상황에서 먼저 떠올리면 좋은 STL입니다.